



DARTMOUTH

Machine Learning for Visual SLAM

from Scene Representation Perspective

Yewei Huang

COSC 81/281

Reality and Robotics Lab

Department of Computer Science

Dartmouth College

Review: What SLAM does?

In Simultaneous localization and mapping (**SLAM**), a mobile robot maps an **un-known** environment, while using **relative** measurements of that environment as the basis for **localizing** itself.



Random robot vacuum
from Internet



Core Modules:

- **State Estimation**
- **Loop Closure**
- **Optimization**

Review: Classical SLAM pipeline recap

Core Modules:

- State Estimation
- Loop Closure
- Optimization

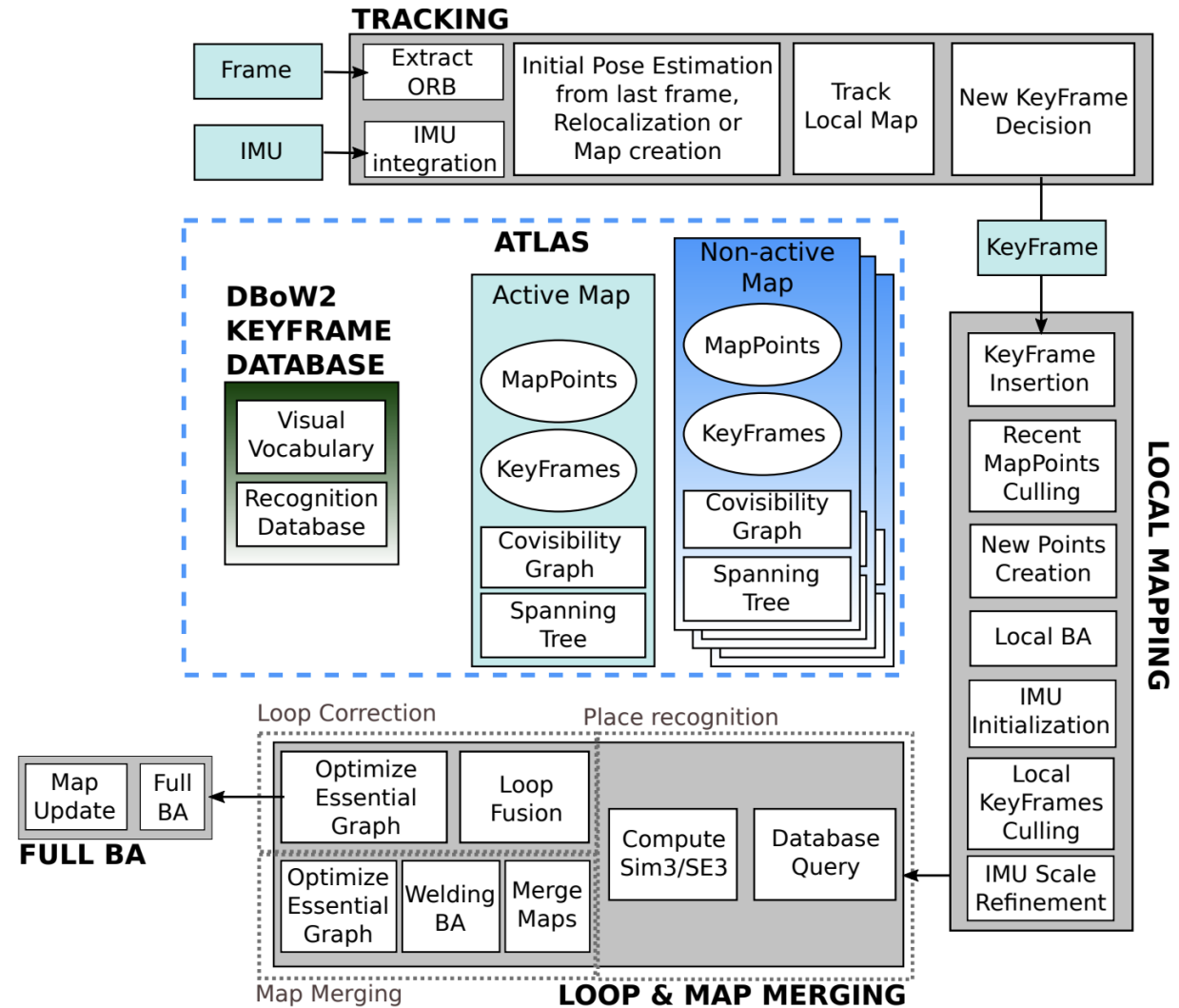
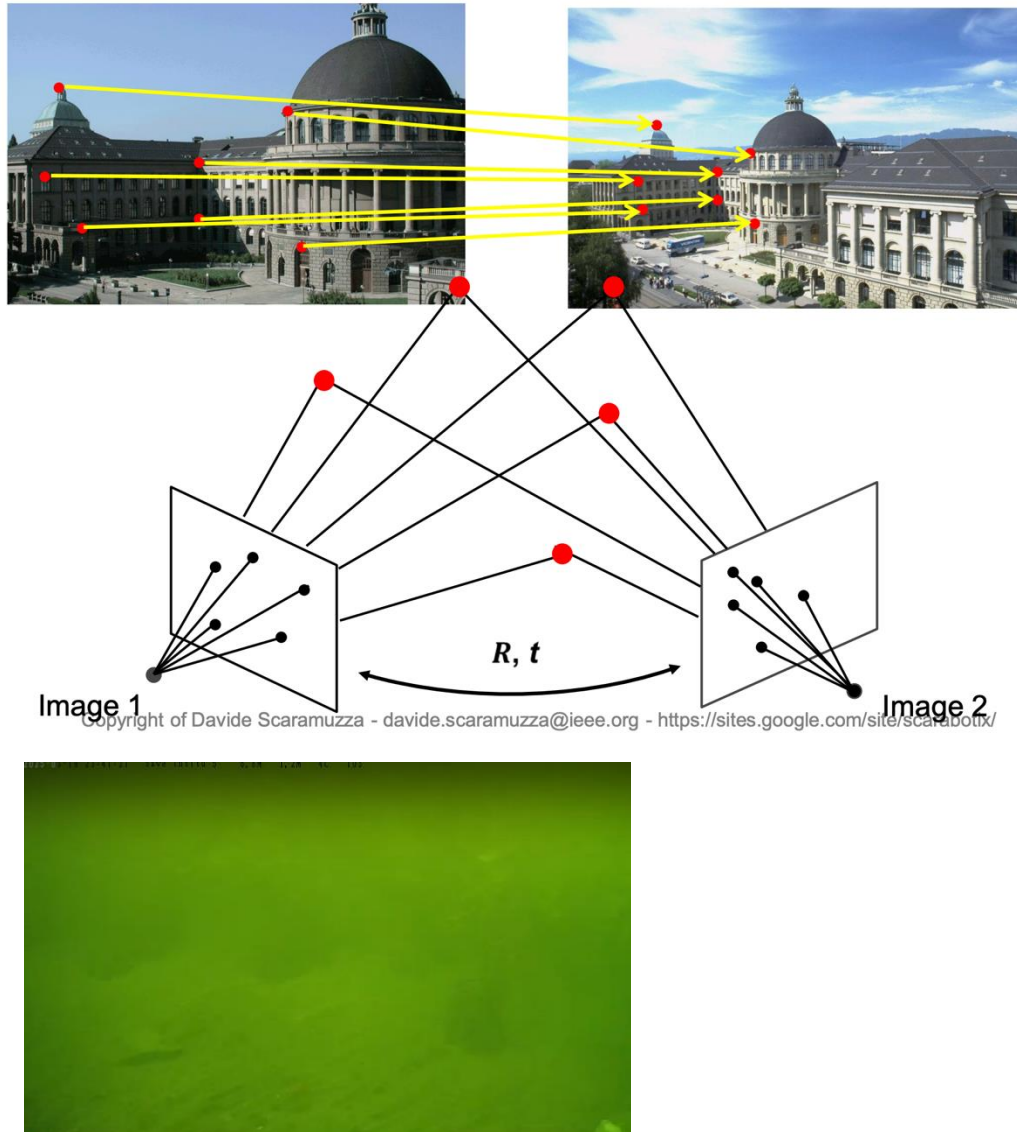


Figure 1: Main system components of ORB-SLAM3.

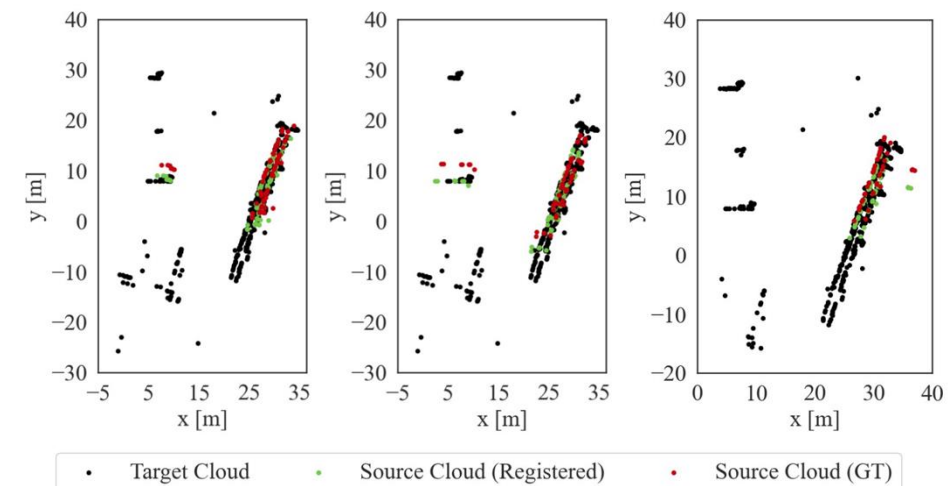
Question: Why add learning?



Classical SLAM:

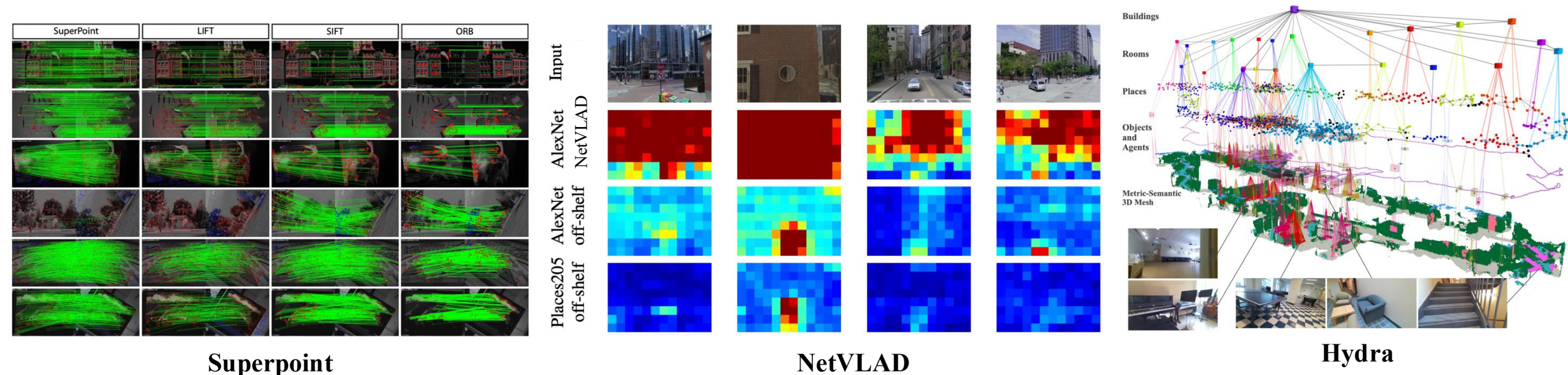
- **State Estimation:** illumination change, viewpoint change, turbidity ...
- **Loop Closure:** ambiguous scenes
- **Optimization:** Long-term data management

A lot of parameter tunings!



Question: Why add learning?

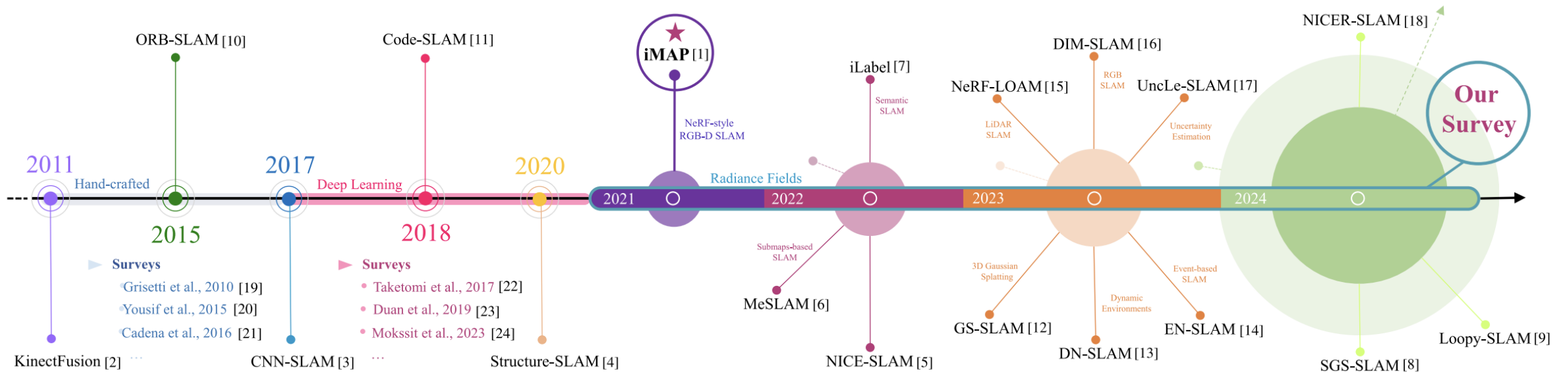
- Improve features, matching, and place recognition
- Learn priors about scene structure and sensor behavior
- Increase robustness under appearance change, ambiguity, and noise
- Build richer maps with semantics, objects, or neural representations



Timeline of SLAM

SplaTAM: Gaussian Splatting SLAM (2024)

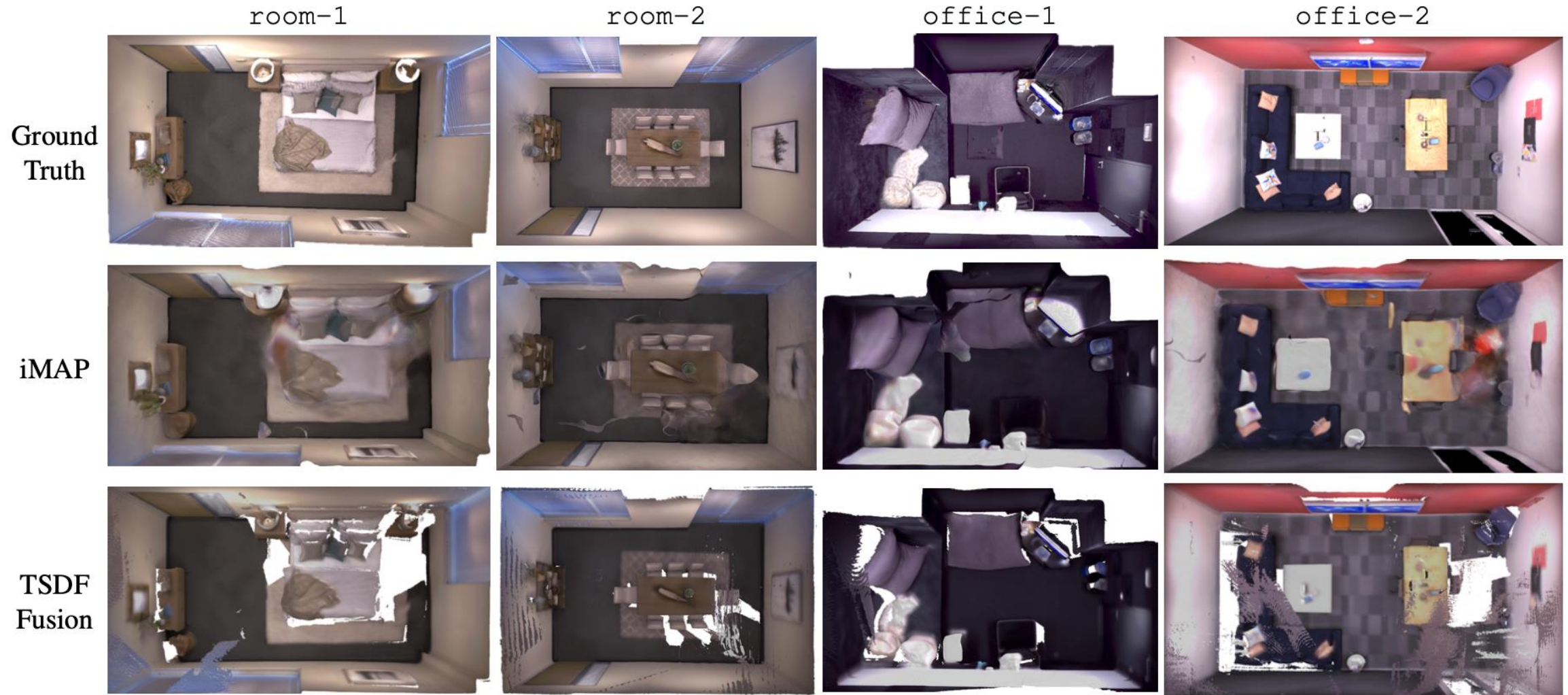
iMAP: early neural implicit / radiance-field SLAM (2021)



Tosi, Fabio, et al. "How nerfs and 3d gaussian splatting are reshaping slam: a survey." *IEEE Transactions on Robotics* (2026).

iMAP: Implicit Mapping and Positioning in Real-Time

- Input: rgb-d images
- Output: SLAM state estimation + implicit neural scene representation

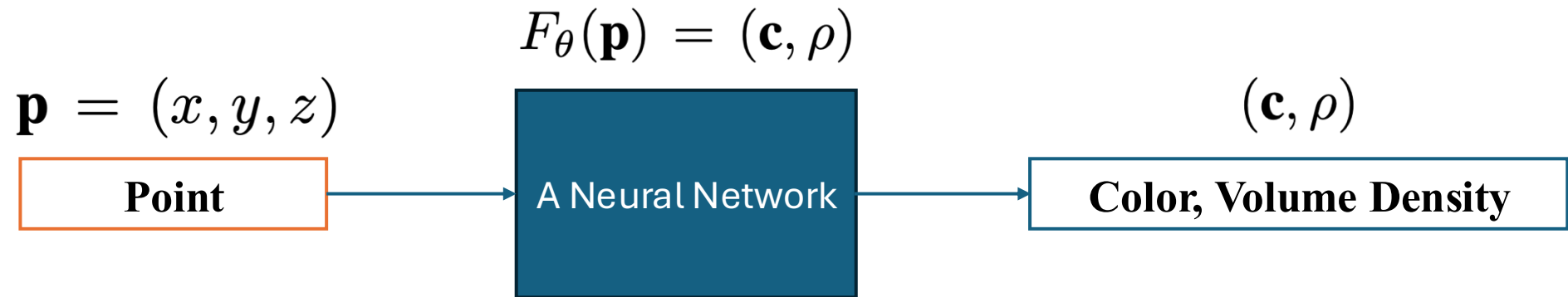


What is Implicit Neural Scene Representation?

ChatGPT ▾

What is Implicit Neural Scene Representation

An implicit neural scene representation is a way to represent a 3D scene as a continuous function learned by a neural network.



iMAP: Implicit Mapping and Positioning in Real-Time

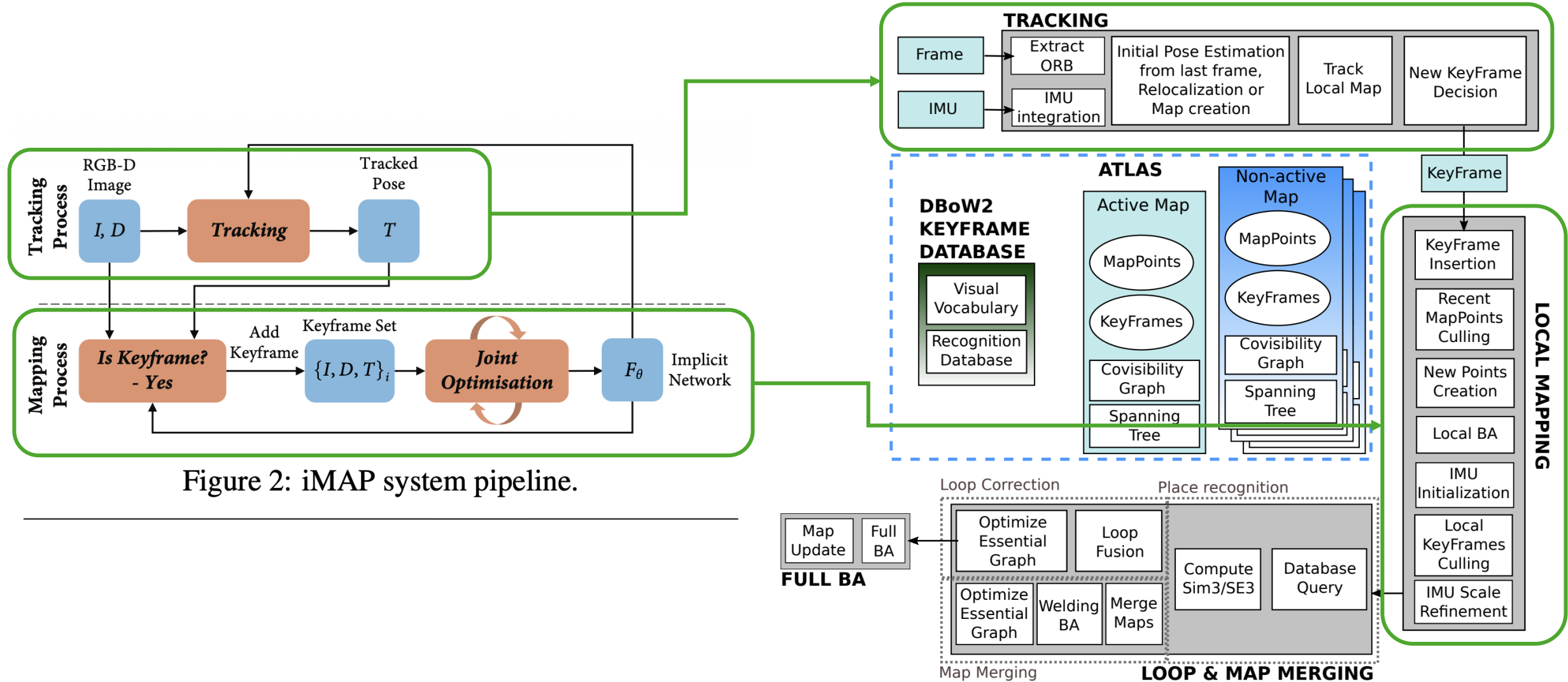


Figure 1: Main system components of ORB-SLAM3.

iMAP: Implicit Mapping and Positioning in Real-Time

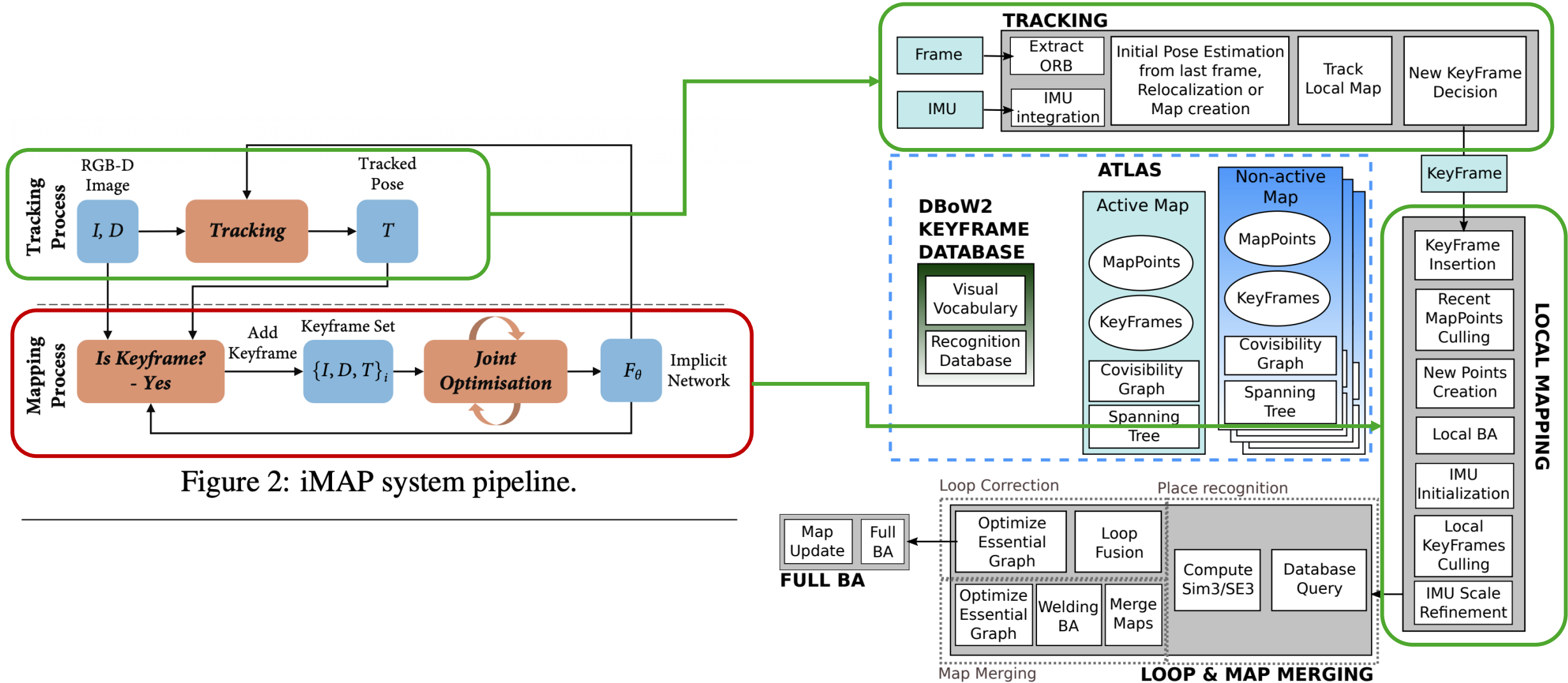


Figure 2: iMAP system pipeline.

Figure 1: Main system components of ORB-SLAM3.

NeRF: Neural Radiance Fields



Neural Radiance Field:

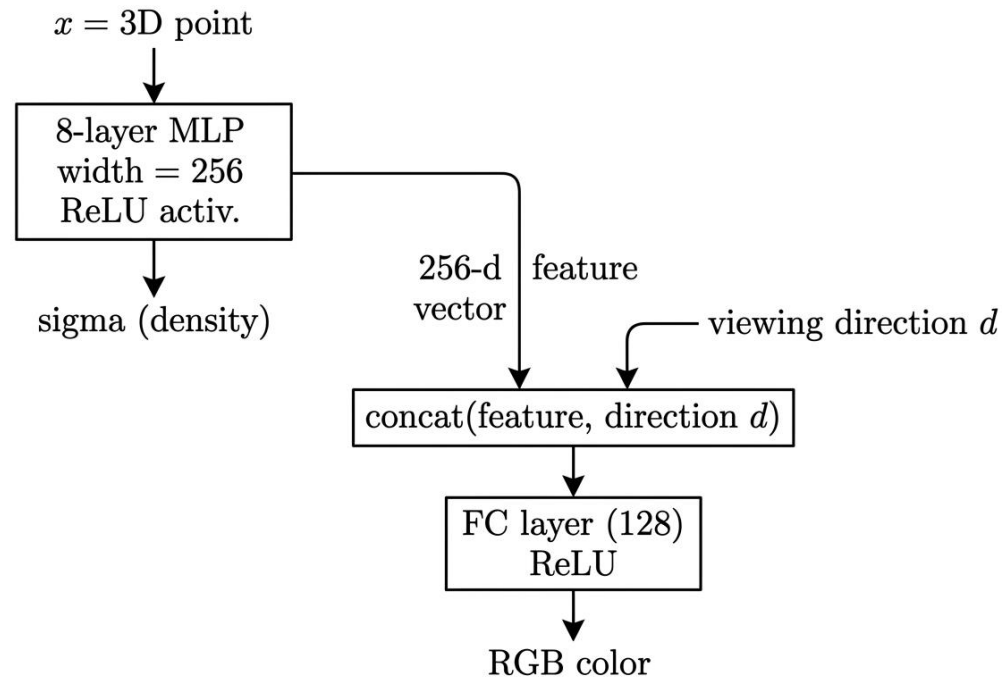
$$F_{\theta}(\mathbf{p}) = (\mathbf{c}, \rho)$$

Input: Point + viewing direction

Output:

Color (r, g, b)

Density (scalar, small: empty/transparent, big: surface)



NeRF: Neural Radiance Fields



Neural Radiance Field:

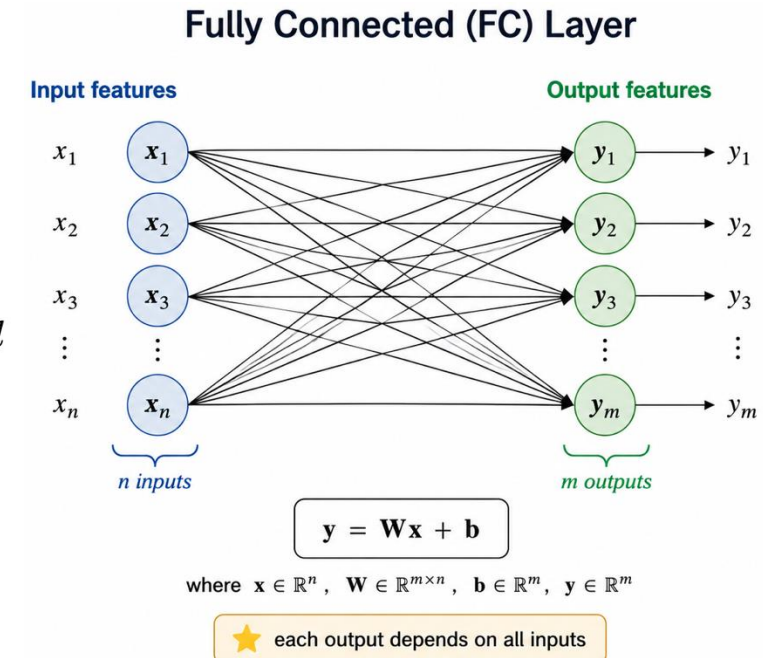
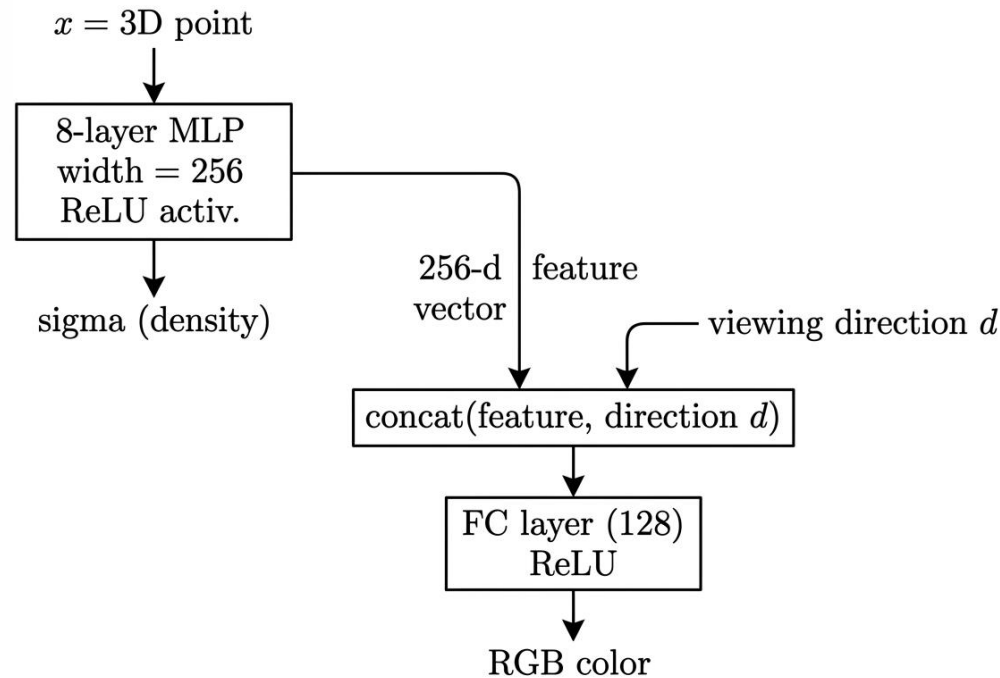
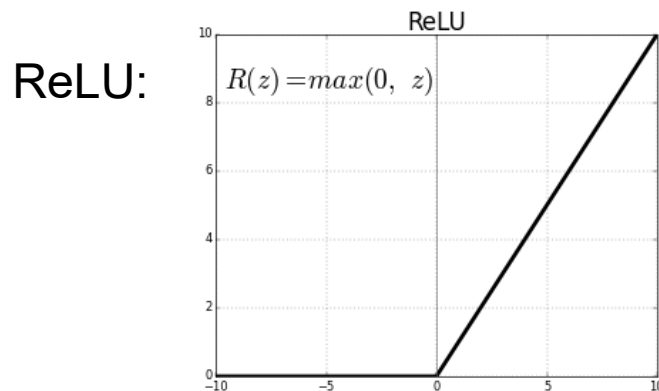
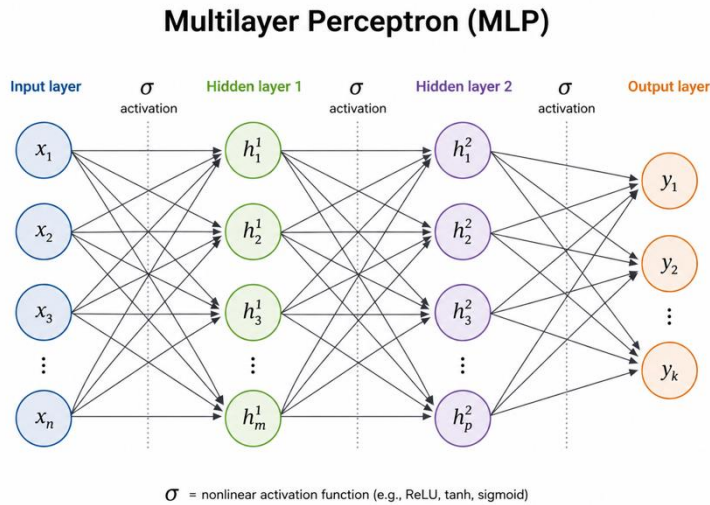
$$F_{\theta}(\mathbf{p}) = (\mathbf{c}, \rho)$$

Input: Point + **viewing direction**

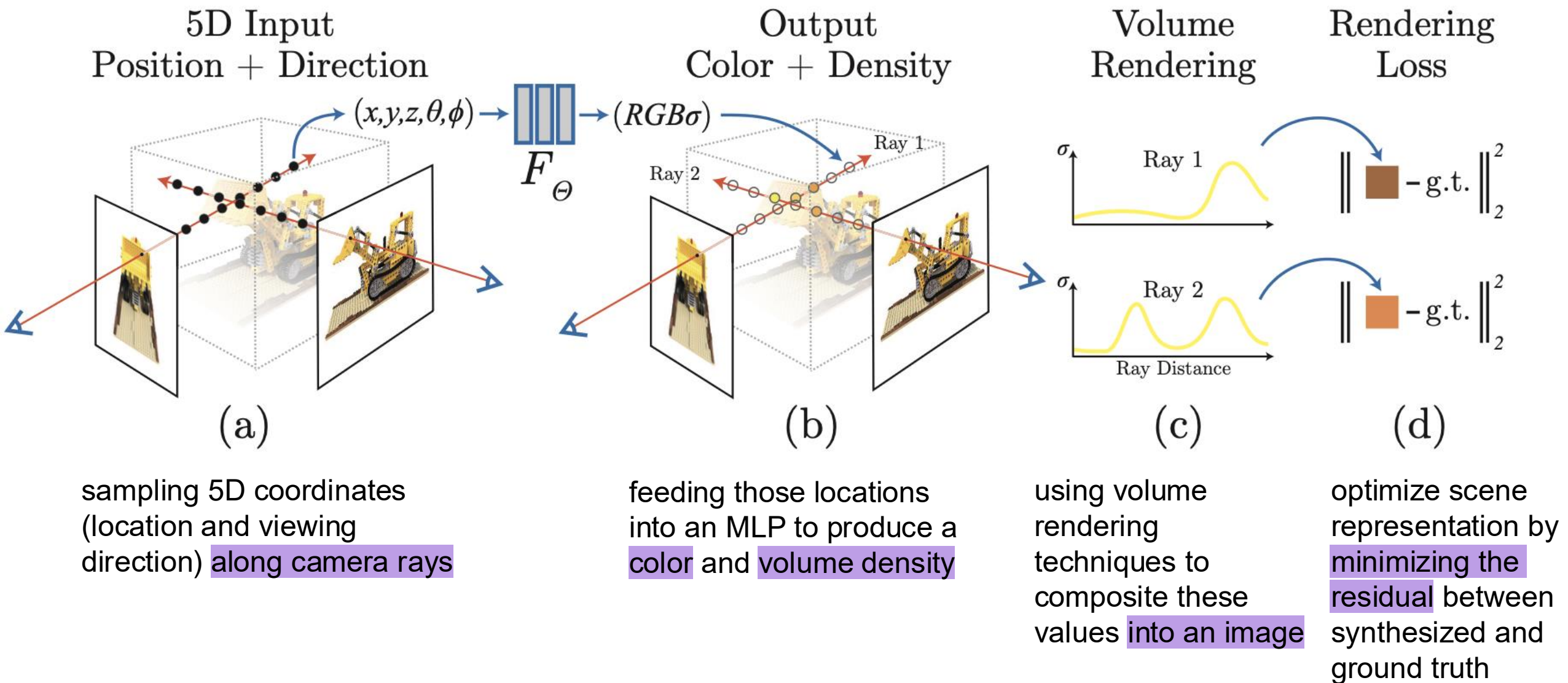
Output:

Color (r, g, b)

Density (scalar, small: empty/transparent, big: surface)



NeRF: Pipeline



Volume Rendering

Continuous

Origin Direction
camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$

$$F_{\theta}(\mathbf{p}) = (\mathbf{c}, \rho)$$

volume density $\sigma(\mathbf{x})$

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt, \text{ where } T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$$

near and far bounds t_n and t_f

Probability: accumulated transmittance along the ray (Visibility)

Discrete

Ray Sampling

$$t_i \sim \mathcal{U}\left[t_n + \frac{i-1}{N}(t_f - t_n), t_n + \frac{i}{N}(t_f - t_n)\right]$$

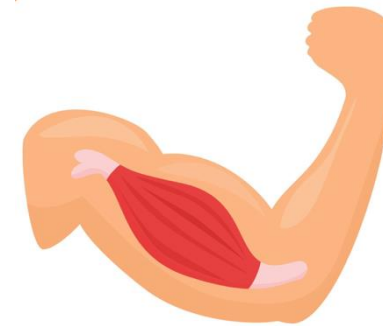
Uniform Distribution

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) \mathbf{c}_i, \text{ where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$$

Probability: local visibility

Defined continuously, computed discretely

Loss Calculation



Densely sampled

$$\mathcal{L} = \sum_{\mathbf{r} \in \mathcal{R}} \left[\left\| \hat{C}_c(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 + \left\| \hat{C}_f(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 \right]$$

coarse volume predicted

ground truth

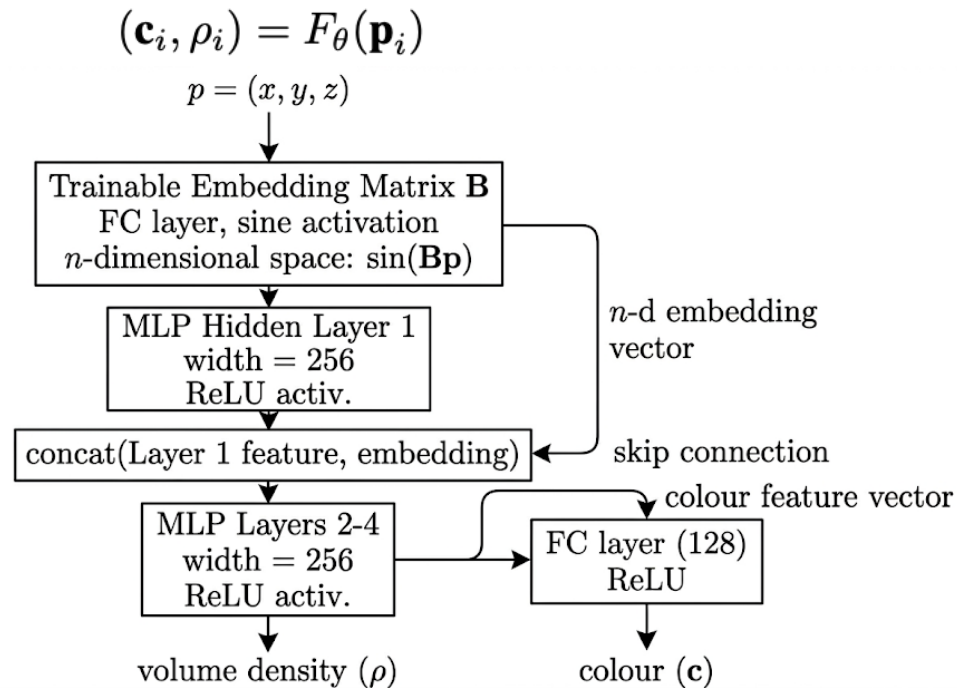
fine volume predicted RGB colors

Uniformly sampled



iMAP: Mapping Process

- What is Implicit Neural Scene Representation?



- How to get this Implicit Neural Scene Representation?

Volume Rendering

Point $\mathbf{r} = T_{WC} K^{-1}[u, v]$

Sampling $\mathbf{p}_i = d_i \mathbf{r}$

Depth $\hat{D}[u, v] = \sum_{i=1}^N w_i d_i, \quad \hat{I}[u, v] = \sum_{i=1}^N w_i \mathbf{c}_i$ **Color**

termination probability $w_i = o_i \prod_{j=1}^{i-1} (1 - o_j)$

activation function $1 - \exp(-\rho_i \delta_i)$

Depth along the ray $d_{i+1} - d_i$

iMAP: Mapping Process

Joint Optimization (Loss Calculation)

- Parameters Optimized:

- Neural Scene Representation Network Parameters
- Camera Poses

- Loss Function:** $\min_{\theta, \{T_i\}} (L_g + \lambda_p L_p)$

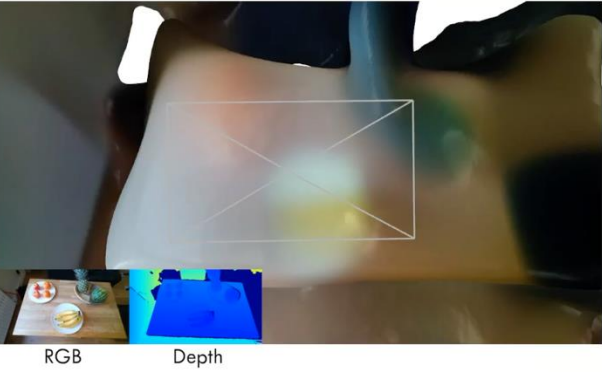
photometric

$L_p = \frac{1}{M} \sum_{i=1}^W \sum_{(u,v) \in s_i} \boxed{e_i^p[u, v]} \quad \text{Color}$
 $\boxed{|I_i[u, v] - \hat{I}_i[u, v]|}$

geometric

$L_g = \frac{1}{M} \sum_{i=1}^W \sum_{(u,v) \in s_i} \frac{\boxed{e_i^g[u, v]}}{\sqrt{\hat{D}_{var}[u, v]}} \quad \text{Depth}$
 $\boxed{|D_i[u, v] - \hat{D}_i[u, v]|}$

Fixed View Reconstruction



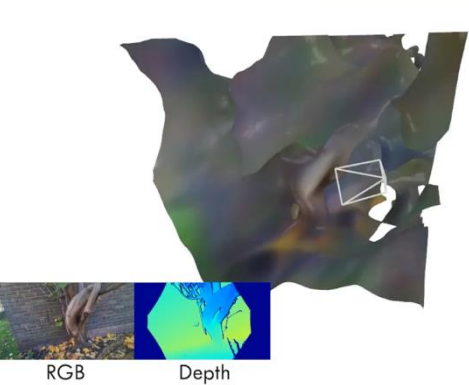
Tracked View Reconstruction



Keyframes



Fixed View Reconstruction



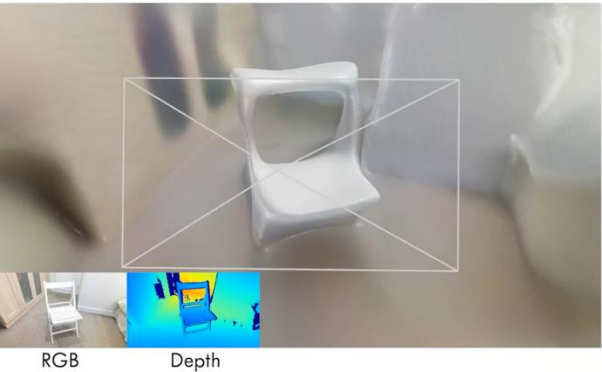
Tracked View Reconstruction



Keyframes



Fixed View Reconstruction



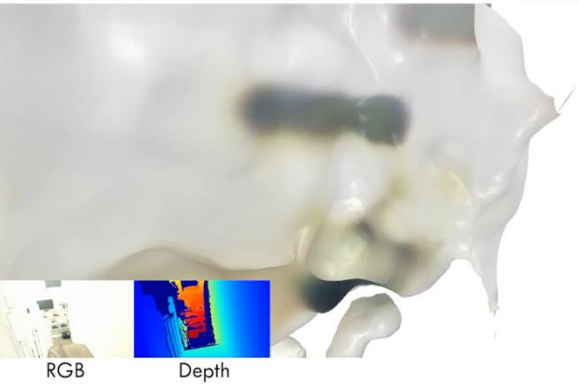
Tracked View Reconstruction



Keyframes



Fixed View Reconstruction



Tracked View Reconstruction



Keyframes



iMAP:

Good:

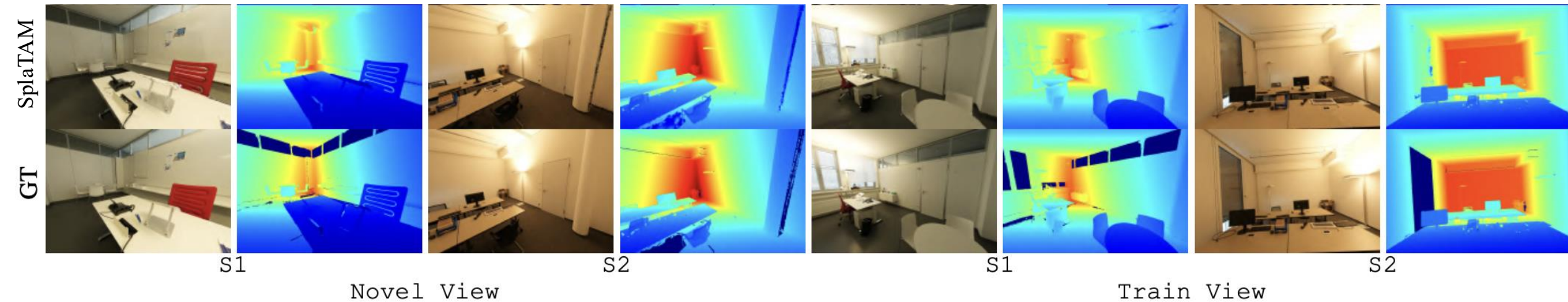
- First real-time online SLAM system using a NeRF-like scene representation
- Memory efficient compared to voxel-based dense SLAM
- Dense and often more complete reconstruction

Bad:

- The state-estimation is less accurate than traditional SLAM methods (ORB-SLAM)
- Speed (tracking 10Hz, mapping 2Hz)
- No direct real-time explicit map visualization
- Oversimplified appearance model

SplaTAM: Splat, Track & Map 3D Gaussians for Dense RGB-D SLAM

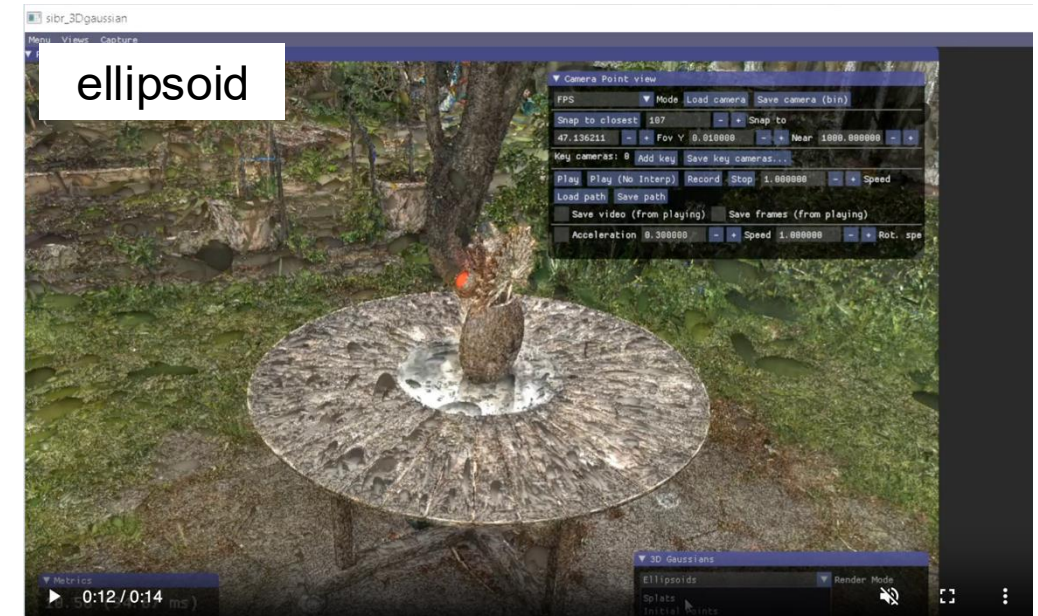
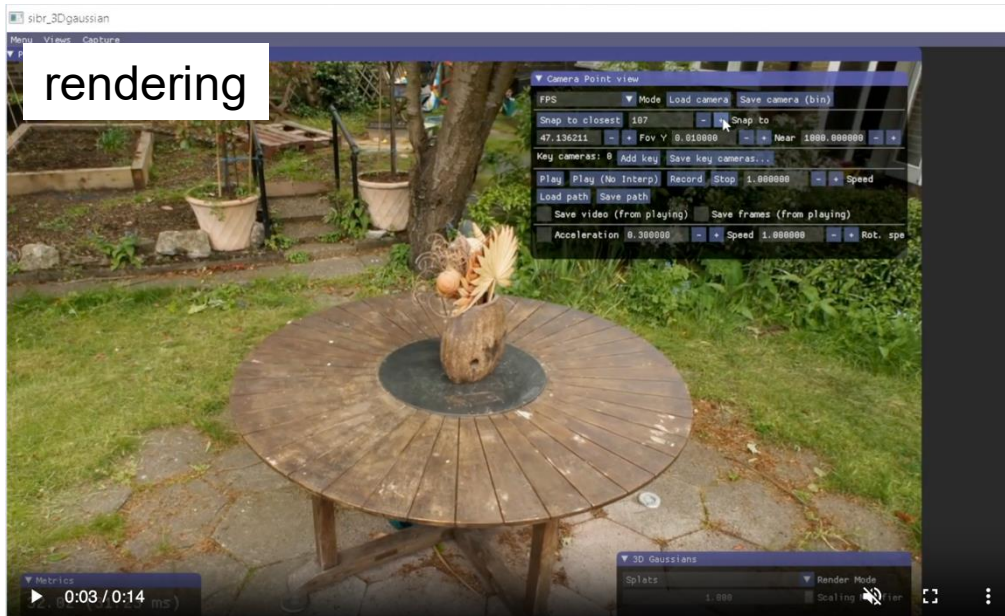
- Input: rgb-d images
- Output: SLAM state estimation + Gaussian Map Representation



What is Gaussian Map Representation?

SplaTAM: “We represent the underlying map of the scene as a set of 3D Gaussians.”

- 3D Gaussians: The Output for 3D GS, a set of ellipsoids (Comparing to 3D voxel map)
 - Each 3D Gaussian (Comparing to each voxel in voxel grid):
 - Mean: Position of the ellipsoid (x, y, z)
 - Covariance: Shape of the ellipsoid (along x, y, z)
 - Color (r, g, b)
 - Opacity
- The “3D Gaussians” is used to render 2D images.



SplaTAM uses a simplified version of 3D Gaussians

- Each 3D Gaussian:

- Mean: Position of the ellipsoid (x, y, z)
- Radius: Shape of the ellipsoid (sphere)
- Color (r, g, b)
- Opacity

$$f(\mathbf{x}) = o \exp\left(-\frac{\|\mathbf{x} - \boldsymbol{\mu}\|^2}{2r^2}\right)$$

Opacity

Mean

Radius

- Rendering

- Color: for a pixel $\mathbf{p} = (u, v)$

$$C(\mathbf{p}) = \sum_{i=1}^n \mathbf{c}_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$$

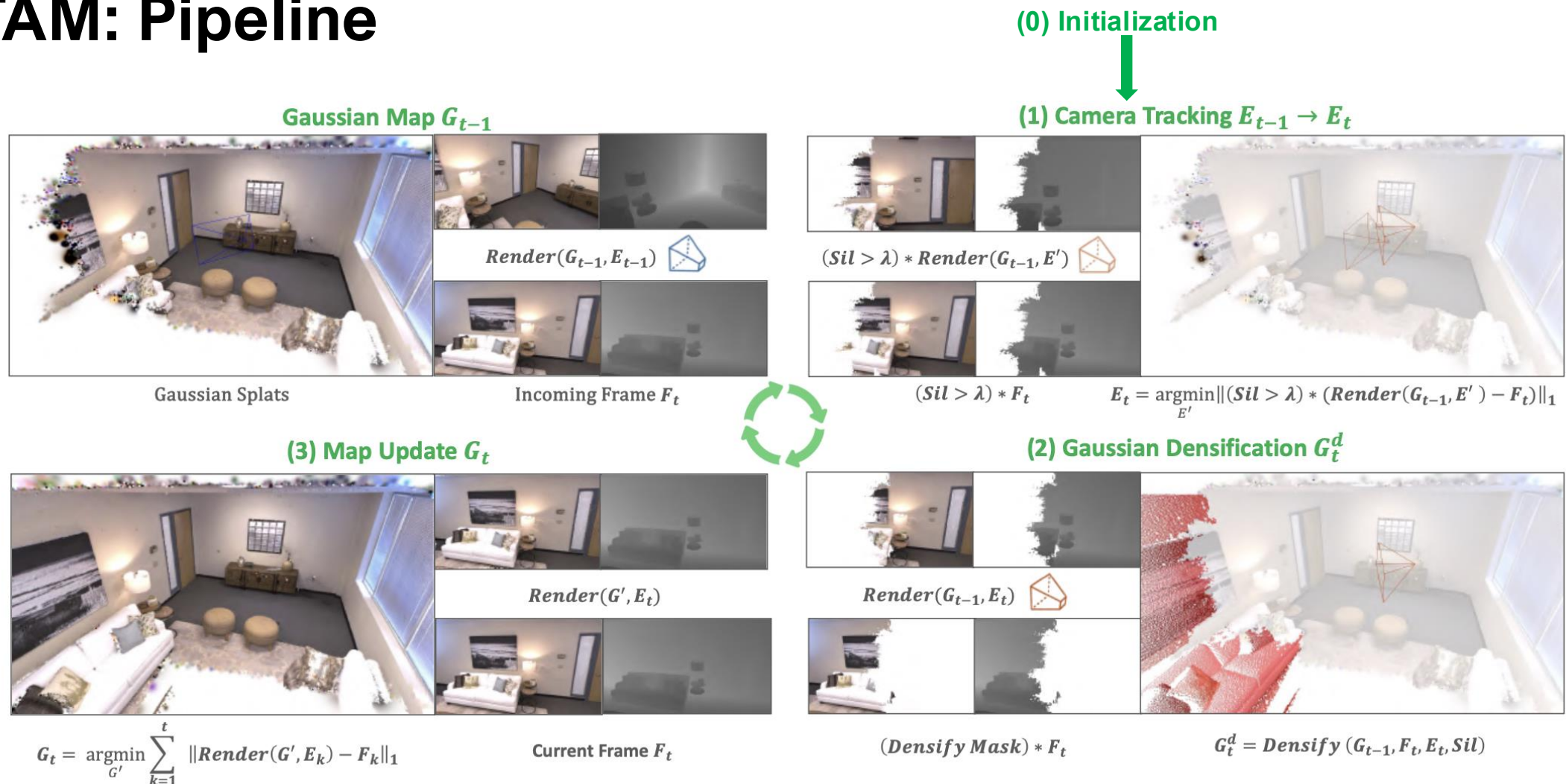
- Depth: $D(\mathbf{p}) = \sum_{i=1}^n d_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$

- Visibility: $S(\mathbf{p}) = \sum_{i=1}^n f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$

2D Gaussian with

$$\underset{\text{intrinsic}}{\boldsymbol{\mu}^{2D}} = K \frac{E_t \boldsymbol{\mu}}{d}, \quad r^{2D} = \frac{fr}{d}, \quad \text{where } \underset{\text{extrinsic}}{d} = (E_t \boldsymbol{\mu})_z.$$

SplaTAM: Pipeline



SplaTAM: Pipeline

- (0) Initialization:

- First frame as initial frame
- Radius of each Gaussian: $r = \frac{D_{GT}}{f}$ depth focal length

- (1) Camera Tracking:

- Initialization with constant kinematic model:

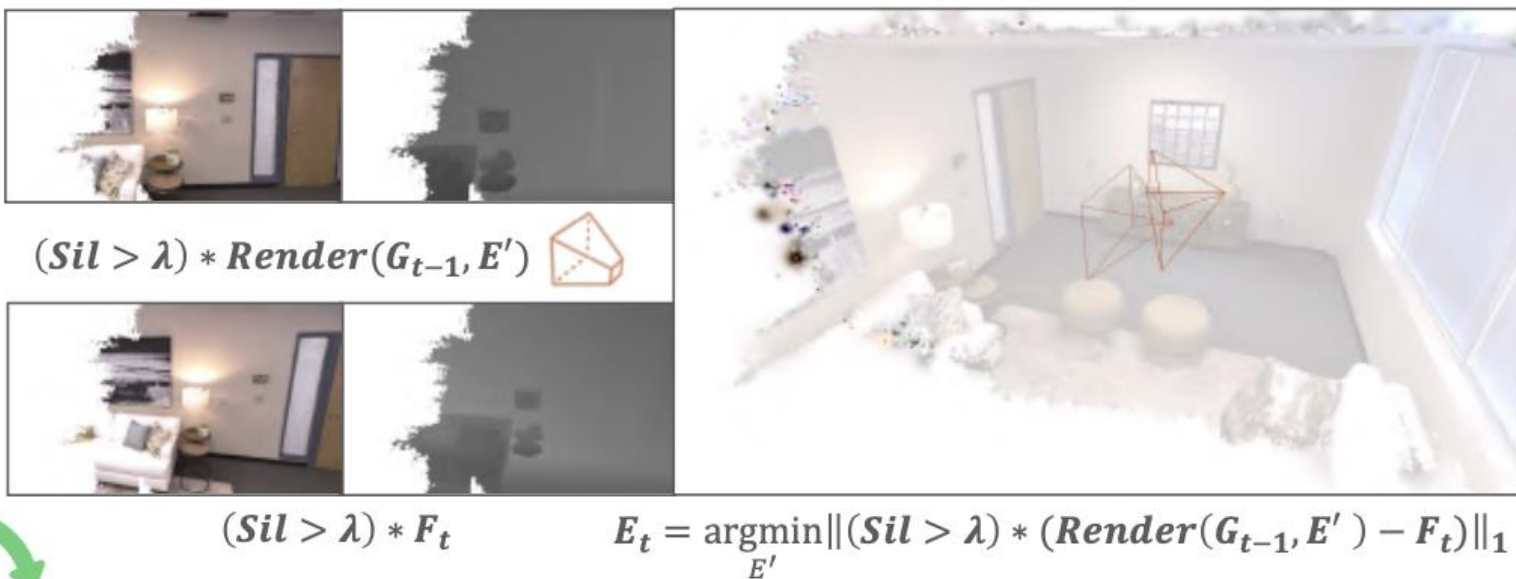
$$E_{t+1} = E_t + (E_t - E_{t-1})$$

- Loss: $L_t = \sum_{\mathbf{p}} \left(S(\mathbf{p}) > 0.99 \right) \left(L_1(D(\mathbf{p})) + 0.5L_1(C(\mathbf{p})) \right)$

- Each 3D Gaussian:
 - Mean: Position of the ellipsoid (x, y, z)
 - Radius: Shape of the ellipsoid (sphere)
 - Color (r, g, b)
 - Opacity
- Rendering
 - Color: for a pixel $\mathbf{p} = (u, v)$

$$C(\mathbf{p}) = \sum_{i=1}^n \mathbf{c}_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$$
 - Depth: $D(\mathbf{p}) = \sum_{i=1}^n d_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$
 - Visibility: $S(\mathbf{p}) = \sum_{i=1}^n f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$

Only the extrinsic is updated!



SplaTAM: Pipeline

- (2) Gaussian Densification
 - Using a mask to decide which part needs to densify

$$M(\mathbf{p}) = \left(S(\mathbf{p}) < 0.5 \right) + \left(D_{GT}(\mathbf{p}) < D(\mathbf{p}) \right) \left(L_1(D(\mathbf{p})) > \lambda \text{MDE} \right)$$

- Add new Gaussians with same step as in (0) initialization



- Each 3D Gaussian:
 - Mean: Position of the ellipsoid (x, y, z)
 - Radius: Shape of the ellipsoid (sphere)
 - Color (r, g, b)
 - Opacity
- Rendering
 - Color: for a pixel $\mathbf{p} = (u, v)$

$$C(\mathbf{p}) = \sum_{i=1}^n \mathbf{c}_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$$
 - Depth: $D(\mathbf{p}) = \sum_{i=1}^n d_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$
 - Visibility: $S(\mathbf{p}) = \sum_{i=1}^n f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$

SplaTAM: Pipeline

- (3) Map Update



Only 3D Gaussians are updated!

- Each 3D Gaussian:
 - Mean: Position of the ellipsoid (x, y, z)
 - Radius: Shape of the ellipsoid (sphere)
 - Color (r, g, b)
 - Opacity
- Rendering
 - Color: for a pixel $\mathbf{p} = (u, v)$

$$C(\mathbf{p}) = \sum_{i=1}^n \mathbf{c}_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$$
 - Depth: $D(\mathbf{p}) = \sum_{i=1}^n d_i f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$
 - Visibility: $S(\mathbf{p}) = \sum_{i=1}^n f_i(\mathbf{p}) \prod_{j=1}^{i-1} (1 - f_j(\mathbf{p}))$



SplaTAM:

Good:

- Accurate online camera pose estimation
- Fast explicit map representation based on 3D Gaussians
- Dense and often more complete reconstruction

Bad:

- Sensitive to motion blur, depth noise, and aggressive rotation
- Potential memory issue for long-term, large-scale globally consistent SLAM

How nerfs and 3d gaussian splatting are reshaping slam?

**Can nerfs and 3d gaussian splatting inspired SLAM
running on embedded systems?**

Can nerfs and 3d gaussian splatting inspired SLAM running on embedded systems? ---- No

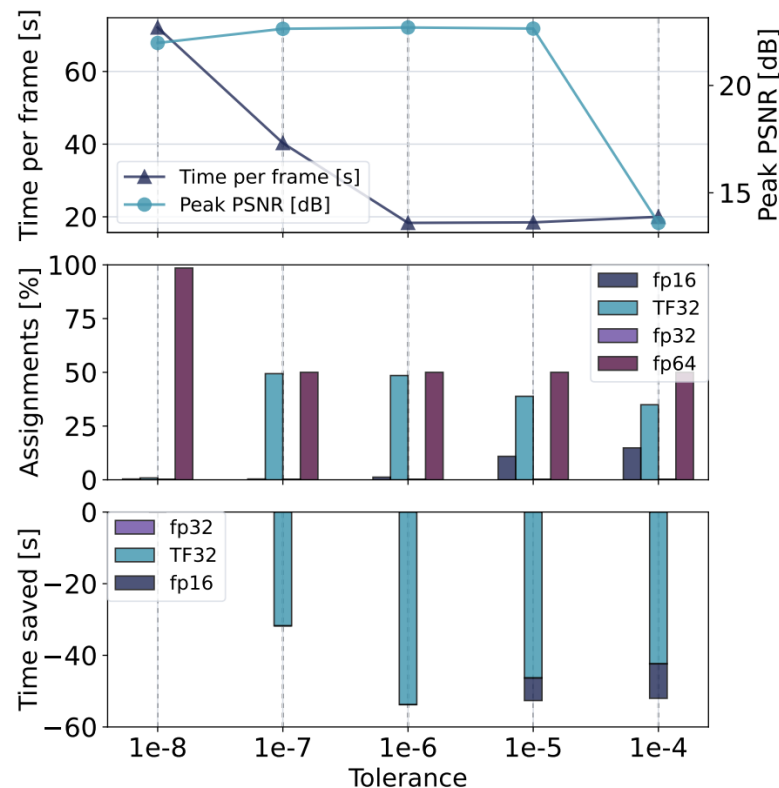


Figure 8. Tolerance sweep for the mixed-precision search on Habitat *Van Gogh Room*.

Table 3. Performance of 3DGS and VBGS training before/after our optimizations on deployment targets. [†]OOM on Jetson.

Implementation	Per-frame average latency	Memory usage
A5000 (3DGS w. replay)	~410 s	4.14 GB
A5000 (VBGS)	70.12 s	9.44 GB
A5000 (Optimized VBGS)	18.33 s	1.11 GB
Jetson (3DGS w. replay)	~3470 s	6.54 GB
Jetson (Baseline VBGS)	n.a. [†]	n.a. [†]
Jetson (Optimized VBGS)	~180 s	2.53 GB

Zaino, Ivan, et al. "Improving Continual Learning for Gaussian Splatting based Environments Reconstruction on Commercial Off-the-Shelf Edge Devices." *arXiv preprint arXiv:2603.08499* (2026).

How nerfs and 3d gaussian splatting are reshaping slam?